```sql
-- =============================================
-- 📊 CUSTOMER CHURN & REVENUE LOSS ANALYSIS
-- =============================================

-- 🔹 1. Create Database
CREATE DATABASE IF NOT EXISTS customer_churn;
USE customer_churn;

-- =============================================
-- 🧹 DATA CLEANING
-- =============================================

-- Check total records
SELECT COUNT(*) AS total_records
FROM synthetic_customer_churn_100k;

-- Identify missing/invalid TotalCharges
SELECT *
FROM synthetic_customer_churn_100k
WHERE TotalCharges IS NULL OR TotalCharges = '';

-- Fix invalid values
SET SQL_SAFE_UPDATES = 0;

UPDATE synthetic_customer_churn_100k
SET TotalCharges = NULL
WHERE TotalCharges = '';
```

```sql
-- Convert TotalCharges to numeric
ALTER TABLE synthetic_customer_churn_100k
MODIFY TotalCharges FLOAT;

-- Check duplicates
SELECT customerID, COUNT(*)
FROM synthetic_customer_churn_100k
GROUP BY customerID
HAVING COUNT(*) > 1;

-- ============================================
-- 📊 CORE METRICS
-- ============================================

-- Churn Rate
SELECT
  COUNT(*) AS total_customers,
  SUM(CASE WHEN Churn = 'Yes' THEN 1 ELSE 0 END) AS churned,
  ROUND(
    SUM(CASE WHEN Churn = 'Yes' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2
  ) AS churn_rate
FROM synthetic_customer_churn_100k;

-- Revenue Loss
SELECT
  ROUND(SUM(TotalCharges),2) AS total_revenue,
  ROUND(SUM(CASE WHEN Churn = 'Yes' THEN TotalCharges ELSE 0 END),2) AS
lost_revenue
```

```sql
FROM synthetic_customer_churn_100k;

-- ============================================
-- 💸 CUSTOMER SEGMENT ANALYSIS
-- ============================================

-- Revenue loss by segment
SELECT
   CASE
      WHEN MonthlyCharges > 80 THEN 'High Value'
      WHEN MonthlyCharges > 50 THEN 'Medium Value'
      ELSE 'Low Value'
   END AS segment,
   SUM(CASE WHEN Churn = 'Yes' THEN TotalCharges ELSE 0 END) AS revenue_lost
FROM synthetic_customer_churn_100k
GROUP BY segment;

-- Ranking segments (Window Function)
SELECT
   segment,
   revenue_lost,
   RANK() OVER (ORDER BY revenue_lost DESC) AS loss_rank
FROM (
   SELECT
      CASE
         WHEN MonthlyCharges > 80 THEN 'High Value'
         WHEN MonthlyCharges > 50 THEN 'Medium Value'
         ELSE 'Low Value'
```

```
        END AS segment,
        SUM(CASE WHEN Churn = 'Yes' THEN TotalCharges ELSE 0 END) AS revenue_lost
    FROM synthetic_customer_churn_100k
    GROUP BY segment
) t;

-- ============================================
-- 📉 CHURN DRIVER ANALYSIS
-- ============================================

-- Churn by contract & payment
SELECT
    Contract,
    PaymentMethod,
    COUNT(*) AS total_customers,
    SUM(CASE WHEN Churn = 'Yes' THEN 1 ELSE 0 END) AS churned,
    ROUND(
        SUM(CASE WHEN Churn = 'Yes' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2
    ) AS churn_rate
FROM synthetic_customer_churn_100k
GROUP BY Contract, PaymentMethod
ORDER BY churn_rate DESC;

-- Churn by tenure (Cohort Analysis)
SELECT
    CASE
        WHEN tenure < 12 THEN '0-1 Year'
        WHEN tenure < 24 THEN '1-2 Years'
```

```
      ELSE '2+ Years'
    END AS tenure_group,
    COUNT(*) AS total,
    SUM(CASE WHEN Churn = 'Yes' THEN 1 ELSE 0 END) AS churned,
    ROUND(
      SUM(CASE WHEN Churn = 'Yes' THEN 1 ELSE 0 END)*100.0/COUNT(*),2
    ) AS churn_rate
FROM synthetic_customer_churn_100k
GROUP BY tenure_group
ORDER BY churn_rate DESC;

-- ============================================
-- 📊 CUSTOMER VALUE ANALYSIS
-- ============================================

-- Estimated CLV
SELECT
    customerID,
    tenure,
    MonthlyCharges,
    (tenure * MonthlyCharges) AS estimated_clv,
    Churn
FROM synthetic_customer_churn_100k;

-- Average CLV by churn
SELECT
    Churn,
    ROUND(AVG(tenure * MonthlyCharges),2) AS avg_clv
```

```sql
FROM synthetic_customer_churn_100k
GROUP BY Churn;

-- ============================================
-- ⚠️ ANOMALY DETECTION
-- ============================================

-- High-paying customers leaving early
SELECT *
FROM synthetic_customer_churn_100k
WHERE MonthlyCharges > 100
AND tenure < 2
AND Churn = 'Yes';

-- ============================================
-- 📈 PARETO ANALYSIS
-- ============================================

SELECT
    customerID,
    TotalCharges,
    SUM(TotalCharges) OVER (ORDER BY TotalCharges DESC) AS running_total,
    SUM(TotalCharges) OVER () AS total_revenue
FROM synthetic_customer_churn_100k;

-- ============================================
-- 💰 REVENUE DISTRIBUTION
-- ============================================
```

```
SELECT
   Churn,
   COUNT(*) AS customers,
   ROUND(SUM(TotalCharges),2) AS revenue
FROM synthetic_customer_churn_100k
GROUP BY Churn;
```